

Évaluation et maximisation de la robustesse d'un réseau routier

De l'importance des cycles dans un graphe

Le pont Morandi



Luca Zennaro / AssociatedPress

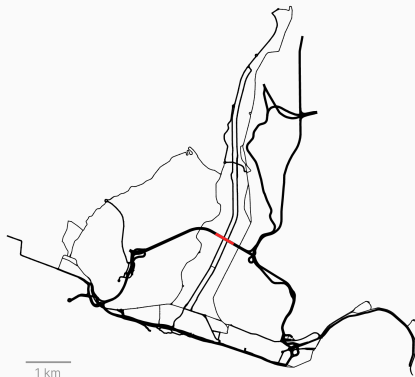


Figure 1 – Effondrement du pont Morandi, le 14/08/2018 à Gênes

Définition (Robustesse)

Capacité d'un réseau routier à limiter les conséquences d'une perturbation sur les usagers, en minimisant les rallongements de temps de trajets lors de la fermeture d'un ou plusieurs axe(s).

- Définir une métrique de robustesse pour un réseau routier, à partir de ses cycles
- L'adapter pour permettre les simulations informatiques

Considérations préliminaires

Réseau routier : graphe $\mathbf{G} = (\mathbf{S}, \mathbf{A}, w)$ non orienté¹

Pour $a = \{i, j\} \in \mathbf{A}$:

$$\underbrace{w(a)}_s = w_{ij} = \underbrace{\ell(a)}_m / \underbrace{v(a)}_{m \cdot s^{-1}}$$

Pour $\{i, j\} \in \mathbf{S}^2 \setminus \mathbf{A}$, on pose $w_{ij} = +\infty$.

1. Modèle : toutes les rues sont à double sens

L'utilité de la matrice d'adjacence

A matrice d'adjacence d'un graphe non pondéré (0 et 1).

$$A^2 = \left(\sum_{k=1}^{|S|} A_{ik} A_{kj} \right)_{1 \leq i, j \leq |S|}$$

→ A_{ij}^n = nombre de chemins de longueur n entre i et j

$\sum_{n=0}^{+\infty} A^n$: nombre de chemins entre chaque paire de sommets

La matrice d'adjacence pondérée

Trajet long $\implies$ peu emprunté

Donc arêtes à grande pondération $\implies$ importance moindre.

Pour cela on remplace la matrice d'adjacence par :

$$\Delta = [e^{-\beta w_{ij}}]_{(i,j)}$$

$\beta > 0$ dépend du graphe. En général, $\beta = \frac{1}{w_{moy}}$ convient.

Une notion de centralité de cycle

Mesure de l'importance d'un cycle dans le graphe

On veut construire une métrique qui témoigne de la centralité d'un cycle dans le graphe

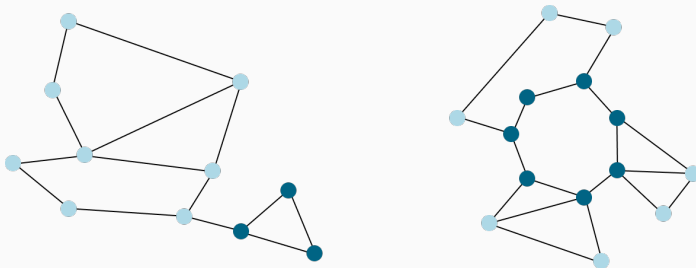


Figure 2 – Un cycle peu central et un cycle très central

Mesure de l'importance d'un cycle dans le graphe

Comment les trajectoires sont modifiées après suppression d'un cycle c ?

Δ_c : matrice d'adjacence de $\mathbf{G}$ sans les arêtes de c .

$$\text{On a : } \underbrace{\sum_{k=0}^{+\infty} \Delta_c^k}_{S_c} = \underbrace{(I_{|S|} - \Delta_c)^{-1}}_{A_c}, \text{ donc :}$$

$$\frac{1}{\det(A_c)} = \det(S_c) = \sum_{\sigma \in S_{|S|}} \varepsilon(\sigma) \prod_{i=1}^n S_{c_{i,\sigma(i)}}$$

Mesure de l'importance d'un cycle dans le graphe

Flux conservatif $\rightarrow$ il se déplace sur des cycles reliés entre eux
(mouvement perpétuel)



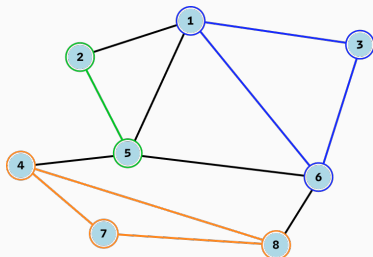
Figure 3 – Dans un réseau fermé, le flux agit comme un fluide (Google Maps)

Mesure de l'importance d'un cycle dans le graphe

Théorème (Décomposition d'une permutation)

Toute permutation se décompose en un produit de cycles à supports disjoints, qui est unique à ordre des facteurs près.

Exemple :



$$\begin{aligned} & (6 \ 5 \ 1 \ 7 \ 2 \ 3 \ 8 \ 4) \\ & = \\ & (1 \ 6 \ 3) \ (2 \ 5) \ (4 \ 7 \ 8) \end{aligned}$$

Mesure de l'importance d'un cycle dans le graphe

Supprimer un cycle $c \implies$ Anéantir les permutations qui reposent sur au moins une arête de c

Arêtes du cycle dans beaucoup de décompositions $\implies$ grand écart entre $\det(A)$ et $\det(A_c)$ ² ($A = I_{|S|} - \Delta$)

On veut donc une métrique $\propto \frac{\det(A_c)}{\det(A)}$

2. $0 \leq \det(A_c) \leq \det(A)$

Métrique finale de centralité d'un cycle

Ainsi, on peut définir la centralité d'un cycle c comme suit :

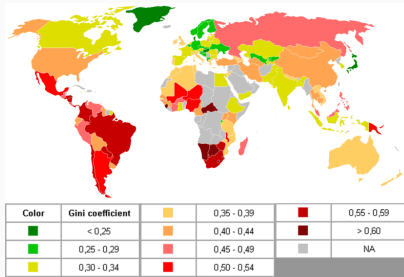
$$\mathcal{C}(c) = 1 - \frac{\det(I_{|S|} - \Delta_c)}{\det(I_{|S|} - \Delta)} \quad (\text{non normalisée})$$

$$\mathcal{C}(c) = \frac{\mathcal{C}(c)}{\sum_{d \in \mathbf{C}} \mathcal{C}(d)} \quad (\text{normalisée})$$

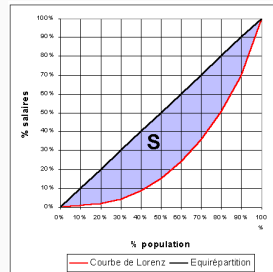
Détermination de la robustesse du réseau routier

Des sciences sociales aux réseaux routiers

On peut avoir une idée de la robustesse d'un réseau grâce à la centralité de ses cycles.



Wikipedia

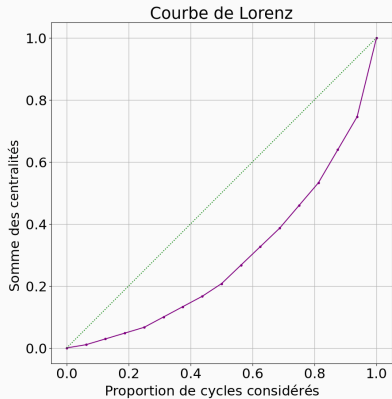
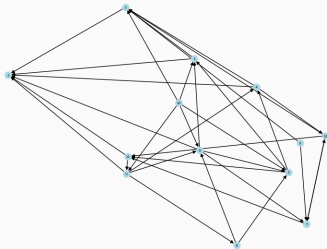


Université Paris Diderot

On va classer les cycles de **G** par centralité croissante, puis observer l'évolution de la somme des centralités ($\mathcal{C}_\Sigma$)

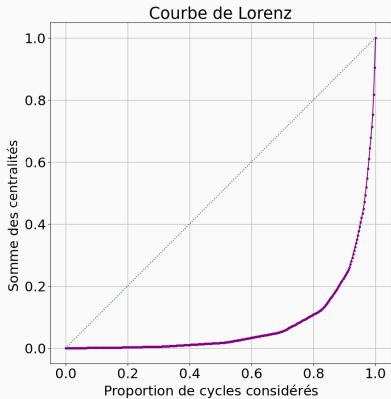
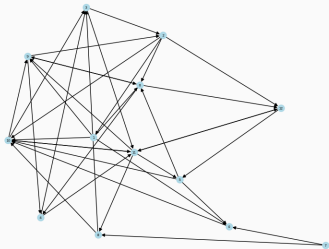
Courbe de Lorenz (1905)

Exemples :



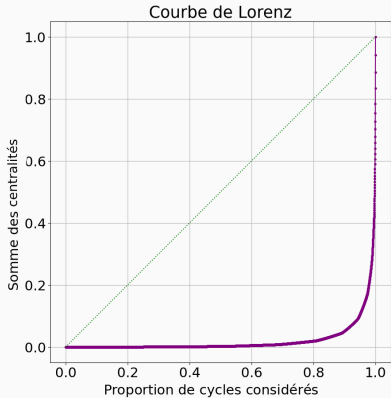
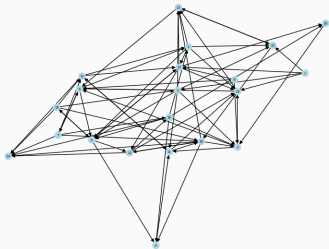
Courbe de Lorenz (1905)

Exemples :



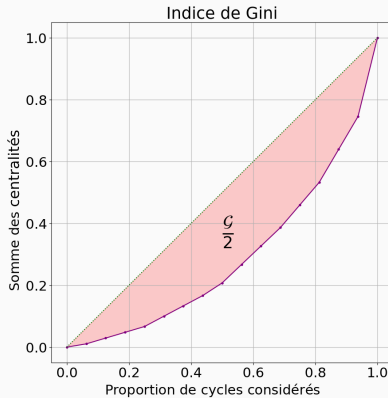
Courbe de Lorenz (1905)

Exemples :



Indice de Gini (1921)

$$\mathcal{G} = 1 - 2 \int_0^1 \mathcal{C}_\Sigma(t) dt$$



Méthodes d'optimisation pour l'implémentation informatique

Indice de Gini $\implies$ calcul sur tous les cycles du graphe

$\rightarrow$ ce nombre peut être énorme

Pour 20 sommets et 30% d'arêtes il est de :

$$\sum_{k=3}^{20} \binom{20}{k} \frac{(k-1)!}{2} (0.3)^k \approx 7.2 \times 10^7 \text{ cycles}$$

Un problème de complexité

En machine : programme long ou infinissable : il faudrait considérer moins de cycles

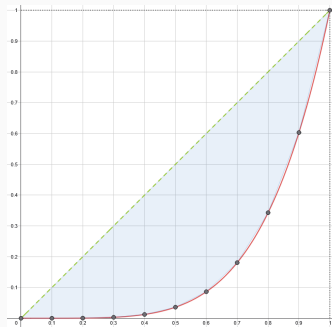
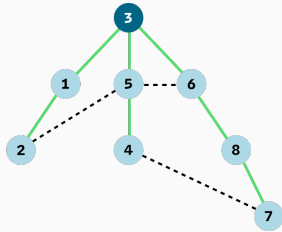
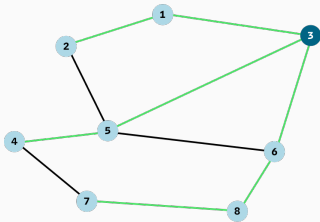


Figure 4 – Une petite proportion de points garde l'information

Quelle liste de cycles choisir ?

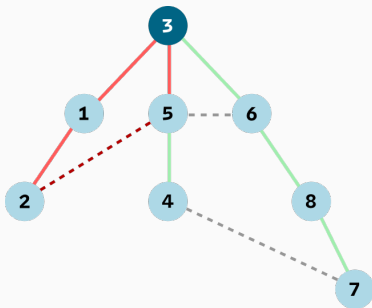
Le choix des cycles fondamentaux - algorithme de Paton (1969)

1. Déterminer un arbre couvrant $\mathcal{A}$ de $\mathbf{G}$



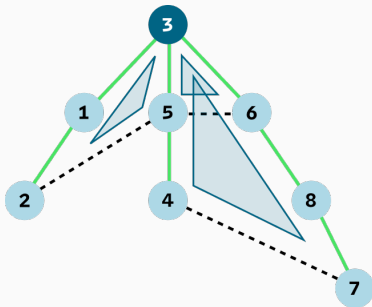
Le choix des cycles fondamentaux - algorithme de Paton (1969)

2. Pour chaque arête de $\mathbf{A} \setminus \mathcal{A}$, il existe un unique chemin dans $\mathcal{A}$ reliant ses deux sommets



Le choix des cycles fondamentaux - algorithme de Paton (1969)

3. On obtient un cycle en ajoutant cette arête au chemin : c'est un **cycle fondamental**



En le faisant sur chaque sommet, on obtient une liste de $|S|(|A| - |S| + 1)$ cycles.

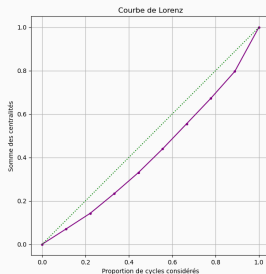
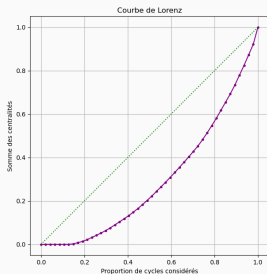
En les combinant, on peut constituer n'importe quel cycle du graphe.

C'est donc une bonne base pour l'approximation de la courbe de Lorenz et de l'indice de Gini.

Approximation par cycles fondamentaux

Cette méthode marche particulièrement bien pour des grands graphes.

→ Pour des petits, on peut faire le calcul sur tous les cycles.



Conclusion

- Métrique construite sur les cycles d'un graphe pondéré
- Les cycles donnent un bon indicateur de robustesse
- Les ajustements faits permettent les calculs sur des plus gros graphes

Après l'effondrement du pont Morandi, Gênes s'inquiète pour son port

Le pont qui s'est effondré était un axe stratégique, emprunté chaque jour par 60 000 voitures et 5 000 camions.



Sandrine Etoa-Andegue - édité par Julien Pasqualini
Radio France

Publié le 20/08/2018 08:23 | Mis à jour le 20/08/2018 13:45

🕒 Temps de lecture : 3min

Après l'effondrement du pont Morandi à Gênes (Italie) le 14 août 2018, de sérieux problèmes logistiques sont à craindre pour le port de la ville, qui est l'un des plus importants de Méditerranée, selon les autorités. Le viaduc autoroutier était en effet un axe stratégique très emprunté dont dépendait l'activité portuaire. Son effondrement oblige les automobilistes à prendre **des déviations qui rallongent le trajet de 150 km.**

"C'est le point d'entrée vers le centre-ville, pour ceux qui travaillent, pour les poids lourds, pour nous tous", explique Enzo, un Gênois rencontré près du pont.

L'idée générale

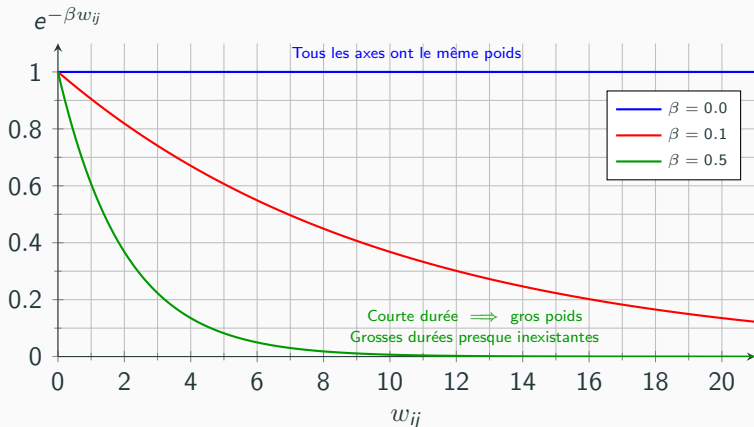


Figure 5 – Différentes valeurs de β

Le choix de β

On doit choisir β pour :

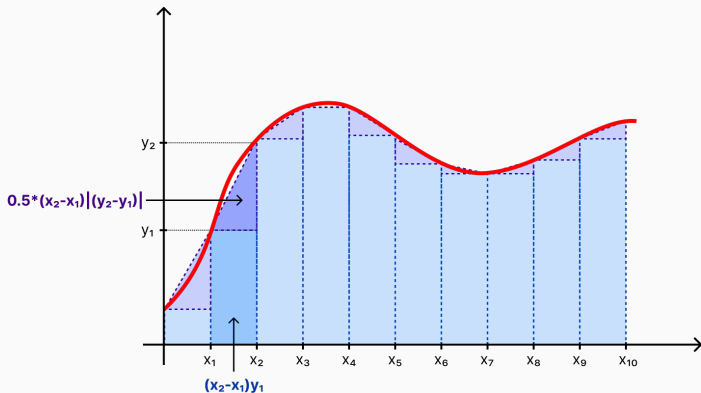
- Donner aux axes rapides une grande importance
- Mais ne pas effacer les axes plus lents

Le β optimal dépend grandement du graphe.

En général, poser $\beta = \frac{1}{w_{\text{moy}}}$ ³ est satisfaisant.

3. $w_{\text{moy}} = \frac{1}{|\mathbf{A}|} \sum_{a \in \mathbf{A}} w(a)$

Approximation de l'indice de Gini



$$\mathcal{A} = \sum_i (x_{i+1} - x_i) \left(y_i + \frac{|y_{i+1} - y_i|}{2} \right)$$

Code source - graphe.py

```
1  #-----          IMPORTS          -----#
2
3
4  import metriques
5
6  import json
7  import networkx as nx
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import random
11 from collections import deque
```

Code source - graphe.py

```
1  #-----  FONCTIONS  GRAPHE  -----#
2
3
4  def from_json(fichier):
5      """
6      Convertit un fichier JSON en graphe NetwokX
7      Le fichier doit être dans le dossier imports, sans l'
8      extension ".json"
9      """
10     with open(f"imports/{fichier}.json", "r", encoding="utf
11               -8") as f:
12         data = json.load(f)
13
14     G = nx.Graph()
15
16     for arete in data.get("aretes", []):
17         G.add_edge(
18             arete["from"],
19             arete["to"],
20             w=arete.get("w", 1.0),
21         )
```

Code source - graphe.py

```
1
2     return G
3
4
5 def to_json(G, nom):
6     """
7     Convertit un graphe NetworkX en fichier JSON
8     Le fichier sera dans le dossier exports
9     """
10    data = {
11        "noeuds": list(G.nodes()),
12        "aretes": [
13            {"from": u, "to": v, "w": attrs.get("w", 1.0)}
14            for u, v, attrs in G.edges(data=True)
15        ],
16    }
17
18    with open(f"exports/{nom}.json", "w", encoding="utf-8")
19        as f:
20        json.dump(data, f, indent=4, ensure_ascii=False)
```

Code source - graphe.py

```
1 def to_img(G, filename="graphe", titre="", taille=(10, 7)):
2     """
3     Exporte un graphe en image
4     Le fichier sera dans le dossier exports
5     """
6     fig, ax = plt.subplots(figsize=taille)
7     pos = nx.spring_layout(G, seed=42)
8
9     nx.draw_networkx_nodes(G, pos, ax=ax, node_color="#4
        C72B0", node_size=600)
10    nx.draw_networkx_labels(G, pos, ax=ax, font_color="white
        ", font_weight="bold")
11    nx.draw_networkx_edges(G, pos, ax=ax, width=1.5, alpha
        =0.7)
12
13    poids_labels = {(u, v): f"{attrs.get('w', 1.0)}" for u,
        v, attrs in G.edges(data=True)}
14    nx.draw_networkx_edge_labels(G, pos, edge_labels=
        poids_labels, ax=ax, font_size=9, bbox=dict(color='#
        fafafa'))
```


Code source - graphe.py

```
1     if titre != "":
2         ax.set_title(titre, fontsize=14, fontweight="bold")
3     ax.axis("off")
4     plt.tight_layout()
5     plt.savefig(f"exports/{filename}.png", facecolor="#
        fafafa", dpi=150, bbox_inches="tight")
6     plt.close(fig)
7
8
9 def matrice_adjacence(G, etat=1, beta=1):
10     """
11     Renvoie la matrice d'adjacence de G
12     Si etat = 1 : matrice pondérée par  $e^{-\beta w_{ij}}$ 
13     Si etat = 2 : matrice non pondérée (1 si l'arête
        existe, 0 sinon)
14     Sinon, matrice pondérée par  $w_{ij}$ 
15     """
16     noeuds = list(G.nodes())
17     n = len(noeuds)
18     index = {noeud: i for i, noeud in enumerate(noeuds)}
```

Code source - graphe.py

```
1     if etat == 1 or etat == 2:
2         matrice = np.zeros((n, n), dtype=float)
3     else:
4         matrice = np.full((n, n), fill_value=np.inf, dtype=
5             float)
6
7     for u, v, attrs in G.edges(data=True):
8         poids = attrs.get("w", 1.0)
9         i, j = index[u], index[v]
10        if etat == 1:
11            poids = np.exp(-(beta * poids))
12        elif etat == 2:
13            poids = 1.0
14        matrice[i, j] = poids
15        matrice[j, i] = poids
16
17    return matrice
```

Code source - graphe.py

```
1 def cycles_fondamentaux_sommet(Delta, s):
2     """
3     Renvoie la liste des cycles fondamentaux de G de matrice
4     d'adjacences Delta à partir du sommet s
5     L'arbre est construit par DFS
6     """
7
8     n = len(Delta)
9
10    # 1. Arbre couvrant
11    racine = s
12    parent = {i: None for i in range(n)}
13    profondeur = {i: 0 for i in range(n)}
14    visites = {racine}
15    file = deque([racine])
16    aretes_arbre = []
17
18    while file:
19        i = file.popleft()
20        for j in range(n):
21            if not np.isinf(Delta[i, j]) and Delta[i, j] !=
22                0 and j not in visites:
```

Code source - graphe.py

```
1         visites.add(j)
2         parent[j] = i
3         profondeur[j] = profondeur[i] + 1
4         file.append(j)
5         aretes_arbre.append(tuple(metriques.
                                   quicksort((i, j))))
6
7     # 2. Détermination des arêtes de G qui ne sont pas dans
      l'arbre
8     aretes_hors_arbre = []
9     set_arbre = set(aretes_arbre)
10    for i in range(n):
11        for j in range(i+1, n):
12            if not np.isinf(Delta[i, j]) and Delta[i, j] !=
              0.:
13                if tuple(metriques.quicksort((i, j))) not in
                  set_arbre:
14                    aretes_hors_arbre.append((i, j))
```

Code source - graphe.py

```
1      # 3. Détermination des cycles pour chaque arête hors de
      l'arbre
2      cycles_fonda = []
3      for (i,j) in aretes_hors_arbre:
4          path_i = [i]
5          curr = i
6          while parent[curr] is not None:
7              curr = parent[curr]
8              path_i.append(curr)
9
10         path_j = [j]
11         curr = j
12         while parent[curr] is not None:
13             curr = parent[curr]
14             path_j.append(curr)
15
16         ancetre_commun = None
17         for node in path_i:
18             if node in path_j:
```

Code source - graphe.py

```
1         ancetre_commun = node
2         break
3
4         partie_i = path_i[:path_i.index(ancetre_commun) + 1]
5         partie_j = path_j[:path_j.index(ancetre_commun) + 1]
6
7         partie_j.reverse()
8
9         cycle = partie_i + partie_j
10        cycles_fonda.append(cycle)
11
12    return cycles_fonda
13
14
15    def liste_cycles_fondamentaux(Delta):
16        """
17        Renvoie la liste des cycles fondamentaux de G de matrice
18        d'adjacences Delta (ensemble des cycles à partir de
19        tous les sommets)
20
21        """
22        n = len(Delta)
```

Code source - graphe.py

```
1     liste = []
2     for i in range(0, n-1):
3         liste.append(cycles_fondamentaux_sommet(Delta, i))
4
5
6 def generer(n=20, d=0.3, filename="graphe"):
7     """
8     Génère un graphe à n sommets, une densité d'arêtes de d,
9     et l'enregistre en json
10    Dans filename, ne pas préciser le .json
11    """
12    edges = []
13    edge_set = set()
14
15    nodes = list(range(n))
16    random.shuffle(nodes)
17    for i in range(1, n):
18        s = nodes[i]
19        t = nodes[random.randint(0, i-1)]
```

Code source - graphe.py

```
1         edges.append({"from": s, "to": t, "w": random.
2             randint(1, 20)})
3
4     for s in range(n):
5         for t in range(s + 1, n):
6             if (s, t) in edge_set:
7                 continue
8             if random.random() < d:
9                 edges.append({"from": s, "to": t, "w":
10                     random.randint(1, 20)})
11                 edge_set.add((s, t))
12
13     with open(f"./exports/exemples/{filename}.json", "w",
14         encoding="utf-8") as f:
15         json.dump({"noeuds": list(range(n)), "aretes": edges
16             }, f, indent=4)
```


Code source - graphe.py

```
1  #----- ALIAS DE FONCTIONS NX -----#
2
3  def nouveau():
4      """Renvoie un graphe non orienté pondéré vide"""
5      return nx.Graph()
6
7
8  def ajouter_arete(G, s, t, w):
9      """
10     Ajoute l'arête (s,t) à G avec la pondération w, modifie
11     la pondération si elle existe déjà
12     """
13     if G.has_edge(s, t):
14         G[s][t]["w"] = w
15     else:
16         G.add_edge(s, t, w=w)
```

Code source - graphe.py

```
1  def supprimer_arete(G, s, t):
2      """
3      Supprime l'arête (s,t) de G
4      """
5      if not G.has_edge(s, t):
6          raise Exception("supprimer_arete: arête inexistante")
7      G.remove_edge(s, t)
8
9
10 def sommets(G):
11     """
12     Renvoie la liste triée des sommets de G
13     """
14     try:
15         return metriques.quicksort(G.nodes())
16     except TypeError:
17         return list(G.nodes())
```

Code source - metriques.py

```
1  #-----          IMPORTS          -----#
2
3
4  import graphe
5  import matrice
6
7  import matplotlib.pyplot as plt
8  import numpy as np
```

Code source - metriques.py

```
1  #-----          CALCULS          -----#
2
3
4  def beta_default(A):
5      """
6      Renvoie 1/w_moy pour la matrice d'adjacence A
7      """
8      somme = 0
9      nb = 0
10     n = len(A)
11     for i in range(n):
12         for j in range(i, n):
13             if not np.isinf(A[i, j]):
14                 nb += 1
15                 somme += A[i, j]
16     return nb/somme
```

Code source - metriques.py

```
1  def quicksort(l):
2      """
3      Applique l'algorithme de tri rapide à une liste
4      """
5      n = len(l)
6
7      if (n == 0 or n == 1):
8          return l
9
10     pivot = l[n//2]
11
12     gauche = [e for e in l if e < pivot]
13     milieu = [e for e in l if e == pivot]
14     droite = [e for e in l if e > pivot]
15
16     return quicksort(gauche) + milieu + quicksort(droite)
```

Code source - metriques.py

```
1  #-----          METRIQUES          -----#
2
3
4  def centralite_non_normalisee(Delta, c, det_I_moins_Delta):
5      """
6      Renvoie la valeur de la centralité non normalisée du
7      cycle c dans G de matrice d'adjacence Delta
8      On passe en argument det(I_n - Delta)
9      """
10     Delta_c = np.copy(Delta)
11     I = matrice.id(Delta.shape[0])
12     n = len(c)
13     for i in range(n-1):
14         Delta_c[c[i]][c[i+1]] = 0.
15         Delta_c[c[i+1]][c[i]] = 0.
16     Delta_c[c[-1]][c[0]] = 0.
17     Delta_c[c[0]][c[-1]] = 0.
```

Code source - metriques.py

```
1     det_I_moins_Delta_c = matrice.det(I - Delta_c)
2
3     if np.isclose(det_I_moins_Delta, 0.0):
4         return 0.0
5
6     return max(0.0, 1.0 - (det_I_moins_Delta_c /
7                           det_I_moins_Delta))
```

Code source - metriques.py

```
1 def centralites_normalisees(Delta, cycles=None):
2     """
3     Renvoie la liste des centralités normalisées des cycles
4     fondamentaux du graphe de matrice d'adjacence Delta
5     """
6     if cycles == None:
7         cycles = graphe.liste_cycles_fondamentaux(Delta)
8     if len(cycles) == 0:
9         return []
10    det_I_moins_Delta = np.linalg.det(matrice.id(Delta.shape
11    [0]) - Delta)
12    centralites_non_norm = []
13    somme_centralites = 0
14    for cycle in cycles:
```


Code source - metriques.py

```
1         centralite = centralite_non_normalisee(Delta, cycle,
           det_I_moins_Delta)
2         centralites_non_norm.append(centralite)
3         somme_centralites += centralite
4     if np.isclose(somme_centralites, 0.0):
5         return [1 / len(centralites_non_norm)] * len(
           centralites_non_norm)
6     return [(1/somme_centralites)*c for c in
           centralites_non_norm]
```

Code source - metriques.py

```
1  def lorenz(Delta, filename="lorenz"):  
2      """  
3      Trace la courbe de Lorenz du graphe de matrice d'  
4      adjacence Delta  
5      Renvoie x et y les coordonnées des points du graphe, né  
6      cessaires pour calculer Gini  
7      """  
8      cycles = graphe.liste_cycles_fondamentaux(Delta)  
9      centralites = centralites_normalisees(Delta, cycles)  
10     n = len(centralites)  
11  
12     if n == 0:  
13         x = [0.0, 1.0]  
14         y = [0.0, 1.0]  
15     else:  
16         centralites_triees = quicksort([max(0.0, c) for c in  
17             centralites])  
18         y = [0.0]  
19         for c in centralites_triees:  
20             y.append(y[-1] + c)
```

Code source - metriques.py

```
1         total = y[-1]
2         y = [val / total for val in y]
3         x = [i / n for i in range(n+1)]
4
5     fig, ax = plt.subplots(figsize=(10, 7))
6     ax.set_aspect("equal", "box")
7     ax.plot(x, y, marker='.', color='purple')
8     ax.plot([0,1], [0, 1], ":g")
9
10    ax.set_xlabel(r'Proportion de cycles considérés')
11    ax.set_ylabel(r'Somme des centralités')
12    ax.set_title("Courbe de Lorenz")
13    ax.set_facecolor("#fafafa")
14    ax.grid(True)
15
16    fname = f"exports/lorenz/{filename}.png"
17
18    plt.savefig(fname, bbox_inches='tight', facecolor="#
        fafafa")
19    plt.close(fig)
20    return x, y
```

Code source - metriques.py

```
1  def gini(x, y):
2      """
3      Renvoie l'indice de Gini du graphe duquel on a calculé x
4      et y dans lorenz
5      """
6      if len(x) < 3 or len(y) < 3:
7          return 0.0
8      g = 0
9      for i in range(1, len(x)-1):
10         g += (x[i+1]-x[i])*(y[i] + 0.5*(abs(y[i+1]-y[i]))))
11     return 1 - 2*g
```

Code source - matrice.py

```
1  #-----          IMPORTS          -----#
2
3
4  import numpy as np
5
6
7
8  #-----          FONCTIONS          -----#
9
10
11 def det(M):
12     """
13     Calculer le déterminant de M (par décomposition LU)
14     """
15     if M.ndim != 2 or M.shape[0] != M.shape[1]:
16         raise Exception("det : matrice non carrée")
17     return np.linalg.det(M)
18
19
20 def id(n):
21     return np.eye(n, dtype=float)
```

Code source - main.py

```
1  #-----          IMPORTS          -----#
2
3
4  import graphe
5  import metriques
6
7  import random
8  import time
```

Code source - main.py

```
1  #-----      FONCTION DE TEST      -----#
2
3  start = time.time()
4
5  for i in range(1, 11):
6
7      print(f"Génération du graphe {i}")
8
9      n = random.randint(5, 30)
10     d = random.randint(15, 40) / 100
11
12     graphe.generer(n, d, f"graphe_{i}")
13
14     print(f"Importation du graphe {i}")
15
16     g = graphe.from_json(f"..../exports/exemples/graphe_{i}")
```

Code source - main.py

```
1     print(f"Calculs sur le graphe {i}")
2
3     Delta_temp = graphe.matrice_adjacence(g)
4     beta = metriques.beta_default(Delta_temp)
5
6     Delta = graphe.matrice_adjacence(g, 1, beta)
7
8     graphe.to_img(g, f"graphe_{i}")
9
10    x, y = metriques.lorenz(Delta, f"lorenz_{i}")
11    print(f"GINI_{i}: {metriques.gini(x, y)}")
12
13    end = time.time()
14
15    print(f"Durée d'exécution: {end-start} secondes ({(end-start)/60} minutes)")
```